ENEL 573 Computer Networks: HW Assignment 4 (Fall 2026)

Name: Temuulen Ankhbayar UCID: _____

Problem 1 (10 points)

An organization with a class B address needs at least 950 subnets. Find the subnet mask and the configuration of each subnetwork.

Step 1. Class B format. A class B address starts with bits **10** and has a 14-bit NET ID and a 16-bit HOST ID (default mask 255.255.0.0, /16). When subnetting, the HOST ID is split into a SUBNET ID (n bit) and a HOST ID (16 - n bit):

```
| 1 0 | NET ID (14 bit) | SUBNET ID (n bit) | HOST ID (16-n bit) |
```

Step 2. Number of subnet bits n. We need $2^n \geq 950$:

```
2^9  =  512 < 950   (not enough)
2^10 = 1024 ≥ 950   →  n = 10 bits subnet ID
HOST ID = 16 - n = 16 - 10 = 6 bits
```

Step 3. Subnet mask. The mask has 1s over the network prefix and the subnet ID (16 + 10 = 26 bits) and 0s over the host ID (6 bits):

```
Default mask: 11111111 11111111 00000000 00000000 = 255.255.0.0   (/16)
Subnet mask : 11111111 11111111 11111111 11000000 = 255.255.255.192 (/26)
```

Subnet mask = 255.255.255.192 (/26 prefix), with 10 subnet bits and 6 host bits.

Step 4. Configuration of each subnetwork.

```
Number of subnets          = 2^10 = 1024 (≥ 950; 74 spare for growth)
Addresses per subnet       = 2^6  = 64
Max hosts attachable/subnet = 2^6 - 2 = 62
  (all-0s host id = subnet address, all-1s host id = broadcast)
```

The question does not give the actual class B network, so I use the class B network from the class notes, **150.100.0.0/16** (10010110 = 150, 01100100 = 100), to show the configuration. The same pattern holds for any class B network a.b.0.0.

Range of addresses for subnet X, where X is the 10-bit subnet ID:

```
Starting address: 10010110 01100100 XXXXXXXX XX000000  = subnet address
Last address    : 10010110 01100100 XXXXXXXX XX111111  = broadcast address
X takes values from 0000000000 to 1111111111 (Subnet #0 to Subnet #1023)
```

Since the 10 subnet bits fill the whole 3rd byte plus the top 2 bits of the 4th byte, for Subnet #k the 3rd byte = floor(k/4) and the 4th byte = 64 × (k mod 4). So:

```
Subnet #k address   = 150.100.floor(k/4).(64·(k mod 4))/26
Subnet #k broadcast = subnet address + 63
```

#	Subnet ID X	Subnet address	First host	Last host	Broadcast
#0	0000000000	150.100.0.0/26	150.100.0.1	150.100.0.62	150.100.0.63
#1	0000000001	150.100.0.64/26	150.100.0.65	150.100.0.126	150.100.0.127
#2	0000000010	150.100.0.128/26	150.100.0.129	150.100.0.190	150.100.0.191

#	Subnet ID X	Subnet address	First host	Last host	Broadcast
#3	0000000011	150.100.0.192/26	150.100.0.193	150.100.0.254	150.100.0.255
#4	0000000100	150.100.1.0/26	150.100.1.1	150.100.1.62	150.100.1.63
#5	0000000101	150.100.1.64/26	150.100.1.65	150.100.1.126	150.100.1.127
…	...	…	…	…	…
#949	1110110101	150.100.237.64/26	150.100.237.65	150.100.237.126	150.100.237.127
#950	1110110110	150.100.237.128/26	150.100.237.129	150.100.237.190	150.100.237.191
…	...	…	…	…	…
#1022	1111111110	150.100.255.128/26	150.100.255.129	150.100.255.190	150.100.255.191
#1023	1111111111	150.100.255.192/26	150.100.255.193	150.100.255.254	150.100.255.255

Subnets #0 to #949 give the 950 required subnets; #950 to #1023 remain free. Each subnet has 64 addresses, of which 62 can be assigned to hosts. To the rest of the Internet the organization still looks like one network, 150.100.0.0/16 (external routers use the network address; internal routers use the /26 subnet addresses).

Problem 2 (10 points)

Find the subnetwork address and the host id for the following: IP address: 120.14.122.16, Mask: 255.255.192.0

Class. First byte 120 is in 0 to 127 (first bit 0), so this is a **class A** address with default mask 255.0.0.0 (/8). The given mask 255.255.192.0 has 8 + 8 + 2 = 18 ones (/18), so it is a subnet mask with 18 - 8 = 10 subnet bits and 32 - 18 = 14 host bits.

```
| 0 | NET ID (7 bit) | SUBNET ID (10 bit) | HOST ID (14 bit) |
```

Subnet address. Subnet mask bitwise ANDed with the IP address gives the subnet address (host part set to 0):

```
IP address : 01111000 00001110 01111010 00010000 = 120.14.122.16
Subnet mask: 11111111 11111111 11000000 00000000 = 255.255.192.0
AND        : 01111000 00001110 01000000 00000000 = 120.14.64.0
```

Only the 3rd byte needs work: 122 AND 192 = 01111010 AND 11000000 = 01000000 = 64.

Host id. The host id is the part of the address under the 0s of the mask (IP address AND the complement of the mask):

```
IP address : 01111000 00001110 01111010 00010000
NOT mask   : 00000000 00000000 00111111 11111111 = 0.0.63.255
AND        : 00000000 00000000 00111010 00010000 = 0.0.58.16
```

Check: subnet address + host id = 120.14.64.0 + 0.0.58.16 = 120.14.122.16 ✓ (122 - 64 = 58).

For completeness: network address (class A default mask) = 120.0.0.0; subnet ID bits = 00001110 01 = Subnet #57; this subnet spans 120.14.64.0 to 120.14.127.255.

Subnet address = 120.14.64.0 (120.14.64.0/18). Host id = 0.0.58.16 (14-bit host id 111010 00010000 = 14864).

Problem 3 (10 points)

Consider sending a 1300-byte datagram into a link that has a MTU of 500 bytes. Which of the following is true? (1) offsets 0, 500/8, 1000/8; (2) offsets 0, 480/8, 960/8; (3) offsets 0, 460/8, 920/8; (4) none of the above.

Assume the typical 20-byte IP header (no options), as in the class example.

```
Data bytes in original datagram = 1300 - 20 = 1280 bytes
Max data per fragment = MTU - header = 500 - 20 = 480 bytes
480 / 8 = 60 → a multiple of 8, so 480 bytes can be used
Data split: 480 + 480 + 320 = 1280 → 3 fragments
Last fragment: 1280 - 2×480 = 320 data, 320 + 20 = 340 bytes
```

Offset is measured in units of 8 bytes and gives where the fragment's data is inserted in the original datagram. In the same form as the class example (ID = X copied into all fragments; flag = more-fragments bit):

Fragment	length	ID	flag	offset	data bytes carried
1	=500	=X	=1	=0	0 to 479
2	=500	=X	=1	=480/8 = 60	480 to 959
3	=340	=X	=0	=960/8 = 120	960 to 1279

Option 1 is wrong because it treats 500 bytes as data per fragment, but the 20-byte header must also fit in the 500-byte MTU. Option 3 is wrong because it subtracts the header twice (500 - 40 = 460) and 460 is not a multiple of 8 (460/8 = 57.5), so it could not even be encoded in the offset field.

Statement 2 is true: three fragments with offsets 0, 480/8 (= 60) and 960/8 (= 120).

Problem 4 (10 points)

An IP datagram is fragmented into 10 fragments, each with an independent loss probability of 1%. Find the probability of receiving the datagram correctly if each fragment is transmitted (a) once; (b) twice.

The datagram can only be reassembled at the destination if **all 10** fragments arrive. Let p = 0.01 be the loss probability of one transmission of a fragment. Since losses are independent, the probabilities multiply.

(a) Each fragment sent once.

```
P(fragment received) = 1 - p = 0.99
P(datagram OK) = (1 - p)^10 = (0.99)^10 = 0.9044
```

(b) Each fragment sent twice. A fragment is lost only if *both* of its copies are lost:

```
P(fragment lost)     = p^2 = (0.01)^2 = 0.0001
P(fragment received) = 1 - p^2 = 0.9999
P(datagram OK) = (1 - p^2)^10 = (0.9999)^10 = 0.9990
```

(a) $P = (0.99)^{10} \approx 0.9044$ (about 90.4%). (b) $P = (1 - 0.01^2)^{10} = (0.9999)^{10} \approx 0.9990$ (about 99.9%).

Sending each fragment twice cuts the probability of losing the datagram from about 9.6% to about 0.1%, at the cost of doubling the bandwidth used.

Problem 5 (10 points)

Nodes A, B, and C attach to the same broadcast LAN. If A sends IP datagrams to B with each frame addressed to B's MAC address, will C's adapter process these frames? Will it pass the IP datagrams to C's network layer? How do the answers change if A sends frames with the MAC broadcast address (ARP request frames)?

Frames addressed to B's MAC address (unicast).

• **Yes, C's adapter processes the frames.** On a broadcast LAN every frame that A transmits reaches every adapter on the LAN, including C's. C's adapter must receive each frame and look at (process) the destination MAC address in the frame header to decide whether the frame is meant for it.

• **No, C's adapter does not pass the IP datagrams up to C's network layer.** The destination MAC address is B's MAC address, which does not match C's own 48-bit MAC address and is not the broadcast address. So C's adapter simply discards the frame. Only B's adapter, whose MAC address matches, extracts the datagram and passes it to the upper (network) layer. This is the whole point of link-layer addressing: frames get from one interface to another physically-connected interface without interrupting the other hosts.

Frames sent to the MAC broadcast address FF-FF-FF-FF-FF-FF (ARP query).

• **Yes, C's adapter processes the frames**, for the same reason as above (every frame reaches every adapter on the broadcast LAN).

• **Now C's adapter does pass the contents up**, because the destination MAC address FF-FF-FF-FF-FF-FF is the broadcast address, which every adapter accepts. As in the class notes, with the ARP query "all machines on LAN receive ARP query". C's adapter hands the encapsulated ARP packet to C's ARP module at the network layer. There, C compares the target IP address in the ARP query (B's IP) with its own IP; since they do not match, C does not reply (only B replies to A, using a unicast frame sent to A's MAC address). Likewise, if A had put ordinary IP datagrams into broadcast frames, C's adapter would pass them up and C's network layer (IP) would discard them because the destination IP address is B's, not C's.

Unicast to B: C's adapter processes (receives and checks) the frames but discards them, so nothing is passed to C's network layer. Broadcast (FF-FF-FF-FF-FF-FF): C's adapter processes the frames and passes the contents up to C's network layer, where they are then ignored because they are not addressed to C's IP address.